Examen práctico — Práctica 1 (Propuesta 3): Purga por prefijo de etiqueta

Duración máxima: 1 h 45 min. Sin Internet ni asistentes de IA.

Dispones de:

- Tu implementación de la práctica 1.
- Las 16 pruebas completas de la práctica.
- Las 10 pruebas del grupo **X** correspondientes a este examen.
- Seminario de C++ en https://www.dlsi.ua.es/asignaturas/eda/seminario-c++-parte1.pdf
- Copia del manual de referencia de C++ en https://www.dlsi.ua.es/asignaturas/eda/cpp

Ejecuta `./run_tests.sh X` para ejecutar las pruebas del examen y ver tu calificación.

Ampliación: purgar etiquetas obsoletas

Con el tiempo el catálogo acumula familias de etiquetas temporales (`tmp1`, `tmpviejo`, `temporal`...). Se quiere poder eliminarlas todas de golpe a partir de un **prefijo** y, además, retirar del catálogo los recursos que se queden sin ninguna etiqueta tras esa limpieza.

Todos los métodos de esta ampliación son **añadidos**: no puedes cambiar ninguna firma de la práctica (los grupos A, B y C deben seguir pasando). Recuerda declarar los métodos nuevos en `Resource.h` y `Catalog.h`.

1. Método `Resource::removeTagsWithPrefix`

```
int removeTagsWithPrefix(const string& prefix);
```

- Normaliza `prefix` a minúsculas, **sin** recortar espacios al principio o al final (igual que `removeTag`). No hace ninguna otra transformación.
- Si el prefijo normalizado está **vacío**: devuelve `0` y no modifica el recurso.
- Elimina **todas** las etiquetas que empiezan por ese prefijo. Una etiqueta exactamente igual al prefijo también empieza por él, así que también se elimina.
- Las etiquetas que quedan siguen en **orden lexicográfico**.
- Devuelve el número de etiquetas eliminadas.

  Atención: eliminar elementos de un `vector` mientras se recorre es una fuente clásica de errores. Piensa bien la estrategia antes de escribir.

2. Método `Catalog::purge`

```
int purge(const string& prefix, int& removedResources);
```

Trabaja en dos fases:

1. Aplica `Resource::removeTagsWithPrefix(prefix)` a **todos** los recursos del catálogo, acumulando cuántas etiquetas se eliminan en total.
2. Elimina del catálogo los recursos que se hayan quedado **sin ninguna etiqueta y que hayan perdido al menos una etiqueta en la fase 1**. Un recurso que ya estaba sin etiquetas **antes** de la purga no se elimina.

- El parámetro de salida `removedResources` recibe el número de **recursos** eliminados.
- El valor de retorno es el número total de **etiquetas** eliminadas. Si una misma etiqueta se elimina de varios recursos distintos, se cuentan todas las veces que se ha eliminado.
- Si el prefijo normalizado está vacío: devuelve `0`, pone `removedResources` a `0` y no modifica nada.
- `removedResources` debe asignarse **siempre**, también cuando no se elimina ningún recurso.

  Atención: aquí vuelves a eliminar elementos de un `vector` mientras lo recorres, esta vez el del propio catálogo. Comprueba que `size()` queda coherente después de la purga.

Casos que debes tener en cuenta: purga repetida con el mismo prefijo (la segunda no elimina nada), purga que vacía el catálogo entero, prefijo que no coincide con ninguna etiqueta y prefijo escrito en mayúsculas.

3. Comando `PURGE <prefix>`

- Imprime `OK <t> <r>`, donde `t` es el número de etiquetas eliminadas y `r` el número de recursos eliminados, separados por un espacio.
- Si falta el argumento: `INVALIDCMD`.

Ejemplo:

```
LOAD data/sample_ok.txt
OK 6
ADD bk07 2022 Temporal tempo
OK
PURGE temp
OK 1 1
GET bk07
NOTFOUND
PURGE cs
OK 2 0
GET bk02
bk02 (2008) Introduction to Algorithms [algorithms]
PURGE
INVALIDCMD
```

(`bk07` sólo tenía la etiqueta `tempo`, así que al eliminarla se queda sin etiquetas y desaparece del catálogo. En cambio `bk02` y `bk05` pierden su etiqueta `cs` pero

conservan otras, por lo que no se eliminan.)

Evaluación

- Nota = número de pruebas pasadas.
- El código debe compilar sin errores ni warnings.
- No se puede modificar la carpeta `tests/`.